

Mobile Application Development with Flutter

Lecture 40. Responsive Layouts in Flutter

What are Responsive Layouts?

- Responsive layouts refers to the ability of an application to adjust its UI components (size, position and visibility) based on screen size, orientations (portrait vs landscape), and devices (phone vs tablet)
- Responsive design focuses on fitting the UI into the space
- Adaptive design ensures the UI is usable and feels native to the platform
- There are six primary techniques and tools used in Flutter to create these layouts

1. Expanded and Flexible Widgets

- These widgets are used inside Row or Column to distribute remaining space proportionally rather than using fixed pixel values
- **Expanded:** Forces a child to fill all available remaining space
- **Flexible:** Allows a child to grow or shrink without forcing it to fill the entire space

Prepared by Ehtisham Rashid

2. Wrap Widget (Auto Flow Layout)

- Wrap automatically moves items to next line when space is not enough

```
Wrap(  
  spacing: 10,  
  runSpacing: 10,  
  children: [  
    Chip(label: Text("Flutter")),  
    Chip(label: Text("Dart")),  
    Chip(label: Text("AI")),  
    Chip(label: Text("Machine Learning")),  
  ],  
)
```

3. MediaQuery (Global Context)

- The MediaQuery class provides global information about the device's screen, such as its total width, height, and current orientation
- **Best for:** Making global layout decisions, such as adjusting font sizes or checking if the device is in landscape mode

- **Example:**

```
double width = MediaQuery.of(context).size.width;
```

```
Container(  
  width: width * 0.5,  
  color: Colors.blue,  
)
```

4. OrientationBuilder

- Specifically designed to handle orientation changes, the OrientationBuilder rebuilds its children whenever the device is rotated
- **Best for:** Switching from a single-column layout in portrait to a multi-column layout in landscape

4. OrientationBuilder

```
OrientationBuilder(  
  builder: (context, orientation) {  
    return GridView.count(  
      // 2 columns in portrait, 4 columns in landscape  
      crossAxisCount: orientation == Orientation.portrait ? 2 : 4,  
      children: List.generate(8, (i) => Card(child: Text('Item $i'))),  
    );  
  },  
)
```

5. LayoutBuilder (Local Constraints)

- The LayoutBuilder widget provides the constraints of its **parent widget** rather than the entire screen
- Used when layout depends on parent constraints

```
LayoutBuilder(  
  builder: (context, constraints) {  
    if (constraints.maxWidth > 600) {  
      return Text("Tablet Layout");  
    } else {  
      return Text("Mobile Layout");  
    }  
  },  
)
```


5. LayoutBuilder (Local Constraints)

```
LayoutBuilder(  
  builder: (context, constraints) {  
    if (constraints.maxWidth > 600) {  
      // Return a side-by-side layout for tablets/web  
      return Row(children: [WidgetA(), WidgetB()]);  
    } else {  
      // Return a stacked layout for mobile  
      return Column(children: [WidgetA(), WidgetB()]);  
    }  
  },  
)
```

6. AspectRatio and FractionallySizedBox

- These helper widgets maintain proportions relative to their parent
- **AspectRatio:** Ensures a widget (like an image or video) keeps a specific width-to-height ratio regardless of screen size
- **FractionallySizedBox:** Sizes its child as a percentage (e.g., 50%) of the available parent width or height

Prepared by Erisham Roshied